

SYSTEM DESIGN CHECKLIST

For engineers with 0 to 2 years
of experience.

12 slides · the questions that
actually decide a design review.

Name the constraints before the technology.

REQUIREMENTS FIRST

Functional: what must it do?

Non-functional: how fast, how available,
how consistent, how cheap?

Write the numbers down before choosing
any tool. 'Fast' is not a requirement.

Constraints choose the architecture.

NAME THESE EIGHT

1. daily active users
2. peak requests per second
3. data size now, and in a year
4. read to write ratio
5. latency budget per request
6. consistency requirement
7. availability target
8. cost ceiling per month

A design without numbers is a guess.

DATA BEFORE CODE

What are the access patterns, not just the entities?

Which queries run hottest, and which indexes serve them?

What is the retention policy?

Where does the data live for compliance?

Schema follows queries, never the reverse.

CACHE WITH INTENT

What exactly are you caching, and why?

Where: client, CDN, app, or database?

How does it invalidate? What is the TTL?

What happens on a cold start or a

cache stampede?

A cache you cannot invalidate is a bug.

Invalidation is the design, caching is the trick.

READS AND WRITES

Reads: replicas, cache, denormalisation.

Writes: sharding key, hot partitions,
backfill strategy, migration plan.

Always ask: what is the sharding key,
and what makes it a hot partition?

The key choice decides the ceiling.

QUEUES AND RETRIES

What work leaves the request path?

Is every consumer idempotent?

Retries with backoff and jitter?

Dead letter queue, and who watches it?

Does ordering matter, and for what?

Assume every job runs twice.

DESIGN FOR BREAKAGE

Timeouts on every outbound call.

Circuit breakers on dependencies.

Graceful degradation: what still works?

Blast radius of one bad deploy?

Rollback plan, and how fast?

Everything fails. Plan the behaviour.

YOU CANNOT FIX WHAT YOU CANNOT SEE

Metrics: latency percentiles, error rate,
saturation, throughput.

Logs with request IDs. Traces across services.

An SLO with an error budget.

Alert on symptoms users feel, not causes.

p99 is the user experience, not the average.

THE TWO QUIET ONES

Auth: who can do what, and where is that checked?

Secrets: storage, rotation, blast radius.

PII: what is stored, and for how long?

Rate limits, and cost per request.

Both are design inputs, not afterthoughts.

THREE QUESTIONS THAT END REVIEWS

1. What breaks first when load doubles?
2. How do we roll this back in ten minutes?
3. What changes if traffic grows 10x?

If you cannot answer, the design is not done.

TRADE-OFFS, NOT TOOLS

Nobody is asking whether you know Kafka.

They are asking whether you know why
you chose it, and what it costs you.

Design is the set of trade-offs you can
defend - not the list of things you used.

Defend the choice, not the logo.